Chapter 12: Physical Storage Systems

Question

What are the main components of a magnetic disk system?

Answer

A magnetic disk system consists of:

- **Platters** – Circular disks coated with magnetic material.
- **Spindle** – Rotates all platters at a constant speed.
- **Read/Write Heads** – Access data on disk surfaces.
- **Arm Assembly** – Moves the heads radially to desired tracks.
- **Disk Controller** – Manages disk operations and communication with the CPU.

Question

Explain the components of disk access time.

Answer

Disk access time consists of:

- **Seek Time:** Time to move the head to the correct track.
- **Rotational Latency:** Time for the desired sector to rotate under the head.
- **Transfer Time:** Time to actually read or write the data.

Total Access Time = Seek Time + Rotational Latency + Transfer Time

Question

SSDs can be used as a storage layer between memory and magnetic disks, with some parts of the database (e.g., some relations) stored on SSDs and the rest on

magnetic disks. Alternatively, SSDs can be used as a buffer or cache for magnetic disks; frequently used blocks would reside on the SSD layer, while infrequently used blocks would reside on magnetic disk.

(a) Which of the two alternatives would you choose if you need to support real-time queries that must be answered within a guaranteed short period of time? Explain why.

(b) Which of the two alternatives would you choose if you had a very large customer relation, where only some disk blocks of the relation are accessed frequently, with other blocks rarely accessed?

Answer

(a) Choice: Store critical data directly on SSDs.

- For real-time queries, predictable and low latency is essential.
- Storing entire relations (or critical ones) on SSDs guarantees consistent fast access without relying on cache hits.
- SSDs have much lower seek and access times than magnetic disks, ensuring query deadlines are met reliably.

(b) Choice: Use SSDs as a cache layer.

- For a large relation with skewed access patterns, caching frequently accessed blocks on SSDs is more cost-effective.
- Rarely accessed blocks can remain on cheaper magnetic disks.
- This hybrid approach improves performance for hot data while minimizing storage cost.

Summary:

- (a) Use SSD as **primary storage layer** for real-time, latency-sensitive data.
- (b) Use SSD as a **cache** for large, partially accessed datasets to balance cost and speed.

Question

Some databases use magnetic disks in a way that only sectors in outer tracks are used, while sectors in inner tracks are left unused. What might be the benefits of

doing so?

Answer

Using only outer tracks increases data transfer rates because outer tracks have longer circumferences, allowing more sectors per track. It also reduces average seek time if frequently accessed data is concentrated on outer tracks, improving overall performance.

Question

Flash storage:

(a) How is the flash translation table, which is used to map logical page numbers to physical page numbers, created in memory?

(b) Suppose you have a 64-gigabyte flash storage system, with a 4096-byte page size. How big would the flash translation table be, assuming each page has a 32-bit address, and the table is stored as an array?

(c) Suggest how to reduce the size of the translation table if very often long ranges of consecutive logical page numbers are mapped to consecutive physical page numbers.

Answer

(a) The flash translation table (FTT) is created in memory as an array or mapping structure where each logical page number (LPN) points to its corresponding physical page number (PPN). The table is initialized when the flash device is mounted or formatted and updated on each write.

(b) **Calculation:**

- Total pages = 64 GB/4 KB = $16,777,216$ pages.
- Each page requires 4 bytes (32 bits) in the FTT.
- Size of FTT = $16,777,216 \times 4$ B ≈ 64 MB.

(c) **Reducing table size:**

- Store only ranges of consecutive mappings, e.g., using a start LPN, start PPN, and length.
- This compresses the table by representing large consecutive blocks with a single entry, reducing memory usage.

Question

Consider the following data and parity-block arrangement on four disks:

Disk 1	Disk 2	Disk 3	Disk 4
B1	B2	B3	B4
P1	B5	P6	B7
B8	P2	B9	B10
...	...	...	...

The B_i represent data blocks; the P_i represent parity blocks. Parity block P_i is the parity block for data blocks B_{4i-3} to B_{4i}.

What, if any, problem might this arrangement present?

Answer

The problem is that P_i and B_{4i-3} are stored on the same disk. If that disk fails, both the parity and one of the data blocks are lost, making reconstruction of B_{4i-3} impossible.

Solution: Distribute parity blocks across different disks from the data blocks they protect, so that loss of a single disk does not result in losing both a data block and its parity.

Question

A database administrator can choose how many disks are organized into a single RAID 5 array. What are the trade-offs between having fewer disks versus more disks, in terms of cost, reliability, performance during failure, and performance during rebuild?

Answer

Factor	Fewer Disks	More Disks
Cost	Lower total hardware and power cost.	Higher total hardware and power cost.
Reliability	Higher reliability; lower chance of second failure during rebuild.	Lower reliability; more disks increase probability of a second failure and longer rebuild exposure.
Performance during failure	Degraded mode affects fewer disks; workload smaller.	Degraded mode affects more disks; reconstruction more intensive and slower.
Performance during rebuild	Faster rebuild, reducing vulnerability period.	Slower rebuild due to more data, increasing risk during rebuild.

Question

A power failure that occurs while a disk block is being written could result in the block being only partially written. Assume that partially written blocks can be detected. An atomic block write is one where either the disk block is fully written or nothing is written (i.e., there are no partial writes). Suggest schemes for getting the effect of atomic block writes with the following RAID schemes. Your schemes should involve work on recovery from failure.

(a) RAID level 1 (mirroring)

(b) RAID level 5 (block interleaved, distributed parity)

Answer

(a) RAID 1 (mirroring):

- Write the block to the primary disk first, then to the mirror.
- If a partial write is detected on either disk, use the complete copy from the other disk to restore it.
- This ensures that after recovery, either both copies are fully written or the original block remains unchanged.

(b) RAID 5 (distributed parity):

- Use a write-ahead or logging technique: first write the old data and old parity to a reserved log area before writing new data and parity.
- After new data and parity are fully written, mark the log entry as committed.
- On recovery, if a block is partially written, reconstruct it using the old data and parity from the log or other disks to restore atomicity.
- This ensures that either the new block and parity are fully applied, or the old consistent state is maintained.

Question

Storing all blocks of a large file on consecutive disk blocks would minimize seeks during sequential file reads. Why is it impractical to do so? What do operating systems do instead, to minimize the number of seeks during sequential reads?

Answer

It is impractical because files grow over time and disk space may be fragmented, making it difficult to find a large consecutive free region.
Solution: Operating systems allocate files in **contiguous extents or multiple large contiguous runs**, allowing sequential blocks to be close together. This reduces seeks while accommodating dynamic file growth and disk fragmentation.

Question

List the physical storage media available on the computers you use routinely. Give the speed with which data can be accessed on each medium.

Answer

- **RAM (Main Memory):** 100 ns access time.
- **SSD (Solid State Drive):** 0.1–0.5 ms access time for random reads.
- **HDD (Magnetic Disk):** 5–15 ms access time for random reads, higher for seek + rotational latency.
- **USB Flash Drive:** 0.1–1 ms (depends on USB version and device).
- **Network Storage (NAS/Remote Disk):** 1–10 ms or more, depending on network speed.

Question

How does the remapping of bad sectors by disk controllers affect data-retrieval rates?

Answer

Remapping bad sectors can slightly reduce data-retrieval rates because:

- The controller must redirect reads/writes from the original location to the spare sector.
- Accesses to remapped sectors may require additional internal lookups or longer seek times.

However, it prevents data loss and maintains overall disk reliability.

Question

Operating systems try to ensure that consecutive blocks of a file are stored on consecutive disk blocks. Why is doing so very important with magnetic disks? If SSDs were used instead, is doing so still important, or is it irrelevant? Explain why.

Answer

Magnetic disks:

- Important because seek time and rotational latency dominate access time.
- Consecutive blocks on consecutive tracks minimize head movement and rotational delays, improving sequential read/write performance.

SSDs:

- Less important because SSDs have no moving parts; access time is nearly uniform for all blocks.
- Logical contiguity does not improve performance, though it may still help with filesystem metadata management or range queries.

Question

RAID systems typically allow you to replace failed disks without stopping access to the system. Thus, the data in the failed disk must be rebuilt and written to the replacement disk while the system is in operation. Which of the RAID levels yields

the least amount of interference between the rebuild and ongoing disk accesses? Explain your answer.

Answer

RAID 1 (mirroring) yields the least interference:

- During a rebuild, the replacement disk is simply copied from the surviving mirror.
- All ongoing read operations can continue from the surviving disk without additional computation.
- Write operations require updates to both the active and rebuilding disks, but reads are unaffected.

  Other RAID levels (e.g., RAID 5):

  - Rebuilding a failed disk requires reading data and parity from multiple disks to compute missing blocks.
  - This increases disk I/O and can significantly interfere with ongoing read-/write operations.

Question

What is scrubbing, in the context of RAID systems, and why is scrubbing important?

Answer

Scrubbing:

- Scrubbing is the process of periodically reading all data and parity blocks in a RAID array to detect and correct latent errors.
- It verifies the integrity of stored data using parity or redundancy information.

Importance:

- Detects silent data corruption before it causes data loss.
- Ensures the array remains consistent, reducing the risk of unrecoverable errors during a disk failure.
- Helps maintain reliability and long-term data integrity.

Question

Suppose you have data that should not be lost on disk failure, and the application is write-intensive. How would you store the data?

Answer

Recommended approach:

- Use **RAID 1 (mirroring)** or **RAID 10 (striped mirrors)** to ensure data redundancy and protection against disk failure.
- These RAID levels provide high write performance compared to parity-based RAID (like RAID 5 or 6), which has extra overhead for parity calculations.
- Ensure **write-back or write-through caching** with battery-backed cache or non-volatile memory to protect in-flight writes.
- Optionally, keep periodic backups for additional fault tolerance.